

Stopping the stop: a self-labeling gate for premature termination in autonomous coding agents

Juan Lescano

29 August 2026

Abstract

A coding agent granted standing permission still ends its turn early: it names the next step instead of taking it, offers a menu instead of choosing, or hands the user a command it could have run. We describe a gate on the agent’s stop path that detects this in three lanes of increasing cost – an audit of the agent’s own escape hatch, 299 pattern labeling functions, and a 9B local judge – and re-wakes the agent when the closing message shows the pattern. Building the gate was straightforward; evaluating it was not, because evaluation needs labels and labels need a reader. We show that two usable labels are already present in the transcript and cost no annotation. Before the gate exists, the developer’s own next message says whether they had to ask for work the agent could have done. After the gate intervenes, the agent’s tool calls before the developer speaks again say whether the intervention bought anything. Measured over 747 closings from 43 real sessions, the first label puts the base rate of premature stopping at 10.4% and the shipped gate’s precision at 0.119 while it fires on 74% of closings: barely above chance, and we report that without softening it. The second label grades 41 interventions and finds 35 of them bought work, which locates the cost precisely – the gate is imprecise about when to speak and largely useful when it does. The labels then pay for themselves again. They retire an exemption the design was confident in, releasing turns that wait on launched work, and they demote the two pattern classes that fire at a third of the base rate. We also report the corpus this was first measured on being contaminated by the system’s own output, and what changed when it was rebuilt. Negative results are reported in full, including three separate instructions a 9B judge would not hold, two failed attempts to obtain a usable confidence signal from it, and a claim-verification lane that found almost nothing to catch.

1 Introduction

A coding agent is given standing permission to act: edit files, run builds, commit. It still stops early. It names the next step instead of taking it, offers a menu instead of picking, reports a result and asks whether to continue, or hands the user a command the agent could have run itself. The user’s reply is then one word. That word costs a round trip, breaks the user’s attention, and carries no information the agent did not already have.

The agent can do the work. Woken with nothing more than “keep going,” it usually finishes what it left. The failure sits in the stopping policy, and it is invisible to the agent, which believes each turn ended reasonably.

This paper describes a gate that sits in the agent’s stop path and re-wakes it when the closing message shows the pattern. That part is straightforward. The part that took the work, and the part we think transfers, is the second loop: how the gate discovers which of its own interventions were wrong, from signals already present in the transcript, with no human labeling.

The headline is uncomfortable and we lead with it. The failure happens on about one closing in ten. The gate fires on three in four and is right about one in eight, which is the rate at which the failure occurs at all, and no lane configuration we measured separates from that rate at 95%. The same transcripts then say the interventions are mostly worth making: 35 of 41 graded blocks bought work. Both statements are measured, neither cancels the other, and the second is the one that keeps the gate switched on.

We contribute:

1. A three-lane gate that spends a regular expression before it spends a model, and a model before it spends the user’s attention (Section 3).
2. A weak-supervision label for premature stopping that requires no annotation: the user’s own next message (Section 4).
3. An evaluation over 747 clean closings from 43 sessions, including an ablation of every lane and every pattern class against that label (Section 5).
4. A self-labeling loop that grades each intervention by what the agent did afterwards, and names patterns for demotion on its own (Section 6).
5. Negative results, in full, including two failed attempts to obtain a usable confidence signal from the judge (Section 7).

2 Background and related work

Where the gate runs. Claude Code exposes a *Stop hook*: a program invoked when the agent finishes a turn. Exit status 0 lets the turn end. Exit status 2 discards the ending and wakes the agent with the hook’s message on standard error. The hook sees the full transcript path, the working directory, and a flag marking whether it is already inside a re-wake chain. It is, in effect, an interposition point on the agent’s stopping decision.

Model-as-judge. Using one language model to score another’s output is now standard practice, and its failure modes are documented: position bias, verbosity bias, and self-preference [8]. Our judge is smaller than the agent it judges by roughly two orders of magnitude, which removes self-preference and forces the prompt to carry the whole task.

Self-critique and refinement. Self-Refine [4] and Reflexion [6] have a model improve its own output from its own feedback. Constitutional AI [1] supplies the critique from written principles. Our setting differs in one respect that matters: the agent has already decided it is done, so a self-critique would be asking the party that made the error to detect it. The gate is deliberately external.

Process supervision. Rewarding intermediate steps rather than final answers improves reliability [3]. Premature stopping is a process defect with a correct final answer – the work the agent did is fine – so it is invisible to outcome supervision.

Weak supervision. Snorkel [5] builds training labels from noisy programmatic labeling functions rather than annotators. Our pattern lane is a set of labeling functions in exactly that sense, and Section 6 adds the piece Snorkel assumes you have: an estimate of each function’s accuracy, obtained here from the agent’s later behaviour.

Calibration. Language models carry some signal about their own correctness [2], and self-consistency across samples is a usable proxy [7]. Section 7 reports that both failed here at 9B scale, and that token log-probabilities did not.

3 System

The gate is eight checkers and one local model behind a single orchestrator: 5389 lines of Python and 18 test files. Three lanes decide, cheapest first, and a lane that decides ends the turn’s evaluation.

3.1 Lane 0: auditing the escape hatch

A closing message that begins with **BLOCKED:** is exempt from every other check. An agent that cannot proceed must be able to say so, and a gate that punishes a genuine blocker teaches the agent to lie about its state.

That exemption is also the cheapest escape in the system, and it was used as one. Lane 0 therefore audits the claim against the transcript rather than the prose. Three signals fail the audit: no tool call at all in the turn that declared the blocker, so nothing was tried; four or more distinct blockers named at once, which is a survey rather than a wall; and three consecutive blocked closings, which is a loop. A blocker that survives the audit passes untouched.

An agent that survives the audit and still needs out is given a release phrase drawn from a cryptographic random source, generated per block and present in no file the agent can read. Quoting it back proves the agent read the current block, and the blocker it accompanies is then audited against what actually ran that turn. The phrase exists because every fixed escape token we tried was eventually emitted from memory rather than from the message.

3.2 Lane 1: patterns as labeling functions

The second lane is 299 regular expressions over the agent’s own words, grouped into nine classes by what the closing is doing rather than by what it says: **ask**, **announce**, **defer**, **wait**, **handoff**, **errand**, **read**, **idle** and **ack**. Appendix A gives the full table. The grouping matters because the loop of Section 6 tunes classes rather than individual expressions: one regular expression rarely fires often enough to be judged on its own.

Two design points matter more than the patterns themselves.

Quoted text does not count. Fenced code blocks, markdown tables, inline backticks, quotation marks and reported speech are stripped before matching. Without this, an agent that describes the gate trips it, and a report containing a table of results is condemned by its own table. This was not hypothetical: it fired on this project’s own status reports twice before the table rule was added.

Patterns carry a confidence. A class written **ask:** decides on its own. A class written **ask?:** raises a doubt and defers to the model. Of 299 patterns, 222 are firm and 77 are doubts. This split is what makes a local model affordable: most closings never reach it, and the ones that do are the ones a regular expression was going to get wrong.

3.3 Lane 2: a small local judge

The third lane is a 9B parameter model served locally through Ollama, pinned by version, answering with exactly one token: **STOP** or **OK**. It runs at temperature 0 with a fixed seed.

The judge exists for one distinction a regular expression cannot make. In this corpus the Spanish word *falta* appears both as “work is missing” and as “it was missing and I fixed it.” No amount of pattern engineering separates those; reading the sentence does.

The judge reports how much probability mass it put on the token it chose, read from the logprobs that accompany every response. That number was recorded from the first verdict and used for nothing for two weeks. It separates cleanly: over the 17 held-out messages the judge labels correctly, its lowest score is 0.846, and every block in the production log scores above 0.5. The one verdict below that threshold was a false accusation, scoring 0.228 on a message that closed cleanly. A STOP below 0.5 is therefore not delivered as a block; it is delivered as the proactive question of Section 3.4, which buys the same second look without telling the agent it left work behind. Guessing costs a look; accusing costs trust.

A judge that cannot be reached is treated the same way. An earlier version blocked every stop when the model was silent, on the argument that a switched-off judge is precisely the failure the gate exists to prevent, and it started the daemon itself to avoid that state. Both halves were wrong in the same direction. Starting a 5.5GB model server because a turn ended is a decision belonging to whoever owns the machine: on 2026-08-29 the gate did it on a busy workstation, which ran out of memory and had to be restarted. The gate now starts nothing, and a silent judge degrades to the pattern lanes plus one proactive question.

Two further implementation details proved necessary rather than cosmetic. First, all judge requests are serialized behind a machine-wide lock: the serving daemon batches concurrent requests, and batching changes the answer, so two test suites running at once produced different verdicts on identical input. Second, when the judge is asked to quote the offending sentence back to the agent, the prompt is strictly extractive and forbids translation; without that, the model paraphrased the sentence into English and the quote failed validation.

3.4 The last question before the turn closes

An OK from the judge used to end the turn. It no longer does, when the transcript can name what is missing. The gate reads the turn’s own tool calls, collects the files it wrote, and asks whether there is a next action available right now, naming the candidates it found: the source file changed with no test written beside it, the code changed with nothing in the documentation moving with it. The turn ends when the agent answers that it looked and found nothing, or when it goes and does the thing it found.

The candidates are read off the record rather than inferred, and a turn that leaves none is released without a question. That gate on the question is measured rather than assumed: a candidate exists on 30% of real turns, and the first blocks of the unconditional version bought no work half the time, the worst conversion of any lane (Section 5).

The asymmetry is deliberate. A block says the agent left work behind, and it is wrong whenever the closing was clean. This question makes no such claim, so it costs one paragraph when the answer is no and buys a whole unit of work when the answer is yes. It is asked once per closing: the state file records that the question was put, and the answer to it is released without a second look.

That property is what makes it the right home for two other cases. A block the judge scores below its confidence floor arrives here instead of as an accusation, and so does a stop the judge never answered because it was switched off. In both the gate is uncertain, and a question is what uncertainty is entitled to.

3.5 The message the agent receives

A block writes one short message to the agent. Its wording went through several revisions and the final form is deliberate: it states that the work already done stands, that permission

was granted in advance and does not expire, and that if anything remains the agent should take the step rather than name it.

The reasoning is mechanical rather than courteous. An agent that reads an accusation spends the woken turn defending itself, which is precisely the outcome the block was meant to prevent. We observed this directly with earlier, blunter wording.

The same reasoning keeps the diagnostics out of the message. Which patterns fired, how confident the judge was, whether the lanes disagreed – all of it is useful for tuning and all of it reads as prosecution evidence. It is written to a private log instead. The agent receives the request; the maintainer receives the numbers. Section 6 is built on that log.

4 Experimental setup

4.1 Corpus

We use every session transcript on one developer’s machine: 66 transcripts spanning 2026-08-03 to 2026-08-29, across several projects. From these we extract *closing pairs*: the agent’s final text message of a turn together with the user’s next message. The first build of this yielded 854 pairs. The corrected readers of Section 5.5 yield 764, of which 747 are neither harness noise nor the gate talking to itself.

The corpus is naturally split by a date. The gate was first installed on 2026-08-28, so of the 43 sessions that carry a closing, 32 are *ungated* - produced with no gate in the stop path - and 11 are *gated*. Ungated transcripts are the ones the gate never influenced. The lane ablations run over the whole clean corpus, because a gated closing is still a closing the developer answered; what the split is for is the era comparison, and for reading the gate’s misses, which is only honest where the gate was not shaping the text.

4.2 A label nobody had to write

Evaluating a stop gate requires knowing which stops were premature. Annotating that by hand is expensive and, worse, is annotated by the same person whose intuitions built the patterns.

The corpus already contains the judgement. When an agent stops early, the user’s next message is a bare push: *dale, seguí, continue, keep going*. It carries no new instruction, because none was needed. We detect it with a closed lexicon anchored to the whole message, so that “continue with the refactor” is not counted and “ok dale” is.

This label is noisy in one direction and clean in the other. A push is strong evidence the turn ended too early. Its absence is weak evidence of the opposite, because a user often absorbs the loss and just answers the question. We therefore report recall against pushes and firing rate, and do not report precision, which this label cannot support.

4.3 Metrics

For each lane configuration we report how many closings it fires on, and what fraction of pushed closings it catches. We deliberately avoid a single combined score: the two costs are not commensurable. A missed premature stop costs the user one round trip. A false block costs the agent a full turn and, if the wording is wrong, poisons the next one.

5 Results

5.1 The corpus

A note on which corpus a number belongs to. This section reports the first pass over 841 labelled closings and then the rebuilt pass over 747 clean ones (Section 5.5 says what was wrong and Table 5 restates the headline figures). Where the two disagree the clean numbers are the answer, and the first pass is kept because the difference between them is one of the results.

The 66 transcripts yield 764 closing pairs, from the 43 sessions that contain a closing at all; the other 23 are sessions with a single turn or none. Closings are long: the median is 1345 characters and the 90th percentile is 3683, which is already a warning about pattern matching, since a longer message gives a fixed pattern set more chances to fire on prose that is not a request.

Seventeen closings (2.0%) are harness error pages rather than messages the agent chose to write: an API error or a usage-limit notice left as the last assistant text. These are excluded from every lane. They matter more than their share suggests: 2 of the 78 pushes follow one directly. The developer was pushing past an infrastructure failure, and a gate that reads those as premature stopping is blocking a session that has already ended.

5.2 What each lane fires on

Lane configuration	Fires on	Share
Firm patterns only	389	52.1%
Any pattern (firm + doubt)	515	68.9%
Judge only	460	61.6%
Firm patterns or judge (shipped)	556	74.4%
Firm patterns and judge	293	39.2%

Table 1: Firing rates over the 747 clean closings, with the pattern set as it stood when the corpus was built.

The shipped configuration fires on nearly three closings in four. That number is the most uncomfortable result here and we report it without softening. It has two readings and we cannot separate them with this data. Either the agent genuinely ends most turns with something nameable left open – which matches the authors’ experience and is why the gate was built – or the gate is far too eager and the tolerable rate is much lower.

What we can say is that the two lanes are not redundant. Firm patterns and the judge agree on only 253 closings, while their union is 620. Each catches a large set the other misses, which is the intended division of labour: patterns catch fixed phrasings cheaply, the judge catches the rest.

5.3 Cost

The judge answers in 0.31 seconds at the median and 0.66 at the 90th percentile, running locally on consumer hardware alongside the agent’s own work. The per-pattern confidence split is what keeps this affordable. On the clean corpus 329 closings carry a firm hit and never pay for a model call, and 187 more carry only a doubt, which is a message raised to the judge instead of being decided by a regular expression.

5.4 Against the push label

We evaluate on the 664 ungated closings, of which 6 non-noise closings were followed by a bare user push.

Lane configuration	Fires on	Pushes caught
Firm patterns only	238	3 of 6
Any pattern	426	6 of 6
Judge only	478	6 of 6
Firm or judge (shipped)	514	6 of 6
Firm and judge	202	3 of 6

Table 2: Recall against the user-push label, ungated closings only.

The shipped configuration catches every push. So does the judge alone, and so does the pattern lane alone once doubts are included. Firm patterns alone catch half.

This is a weak result and we want to be exact about why. Six positives cannot distinguish between configurations that all fire on more than a third of the corpus; a detector that fired on everything would also score 6 of 6. The label establishes that no lane is systematically blind to the failure it targets, and nothing more. Precision is the quantity that matters here and this label cannot measure it, which is the entire reason for the loop in Section 6.

5.5 When the label reads the system’s own output

The first build of this corpus was contaminated, and the way it was contaminated is the more general finding. A weak label built by reading transcripts asks who spoke last. In an agent harness the answer is not obvious, because several things wear the user’s role.

The gate wakes the agent by writing to standard error, and the harness feeds that text back as a plain user string. Read as a turn, it is the gate’s own reminder handed to the push judge as though a person had written it: 120 of the 854 pairs were in that state and 9 of the 123 positives were the system scoring itself. The same string also ended the window in which the self-labelling loop counts tool calls, so every block graded as zero work bought. The report read 14 of 14 blocks bought nothing, a figure that is impossible rather than merely bad, and which no amount of prompt tuning would have explained.

Two further sources wore the same role. A screenshot arrives as a placeholder carrying its pixel dimensions, and an interrupted turn, a context compaction and a usage-limit notice each arrive as a paragraph the harness writes about itself. Together with the wakes, 152 of 753 replies contained no developer words at all, so a fifth of the label was the judge reading boilerplate.

The correction is one definition of a developer turn, shared by every reader. Three components had each written their own and each failed differently, which is the usual shape of this bug: it is not a modelling error and no evaluation of the classifier can surface it, because the classifier is working correctly on inputs that should never have reached it.

One of the three was not a measurement bug at all. The component that detects work still in flight cleared its state on any user turn, so the first block retired the agent’s launched task and every stop after it saw nothing running. The reminder that asks a waiting agent to keep working went silent exactly when it was needed, and it did so in production for as long as the feature had existed. Building the measurement is what found it.

5.6 A stronger label, and what it costs to trust it

Bare pushes are too rare to tune against. But the lexicon was only ever an approximation of the question we care about, which is whether the developer’s reply asked for work the agent could have done on its own.

We therefore ask that question directly, with a second local judge that reads the developer’s reply rather than the agent’s closing. It is given the closing for context and asked a single question: did the developer have to ask for work the agent could have done in the turn that just ended? A reply that pushes, picks from a menu the agent offered, or tells the agent to run what it just described is a push. A reply that supplies information the agent could not have had – a preference, a priority, a fact from outside the repository, a correction – is not.

This judgement is deliberately not the gate’s own. The gate’s judge reads the agent’s closing; this one reads the reply. Grading a gate with the prompt it decides by measures its consistency and nothing else.

Two checks on the label before using it. It recovers 10 of the 11 bare pushes the lexicon found, without being given the lexicon. And on a held-out set of nine hand-written boundary cases – including short, blunt replies that carry real information, which are the ones a length heuristic gets wrong – it scores 9 of 9.

It labels 117 of the 841 non-noise closings as pushes: a base rate of **13.9%**.

5.7 The gate fires far more often than the failure occurs

That base rate is the result that reframes everything above.

Lane configuration	Fires on	Precision	Recall	F1
Base rate (the failure itself)	117	0.139	1.00	—
Firm patterns only	304	0.19	0.50	0.28
Any pattern	539	0.16	0.72	0.26
Judge only	569	0.15	0.75	0.26
Firm or judge (shipped)	624	0.15	0.81	0.26
Firm and judge	249	0.21	0.44	0.28

Table 3: Every configuration against the push label. Precision at or near the base rate means the configuration carries almost no information about which turns ended early.

The shipped configuration fires on 74% of closings and is right 15% of the time, against a base rate of 13.9%. It is barely above chance. Requiring both lanes to agree raises precision to 0.21 and halves recall, the only configuration here that clearly beats the base rate, and it still fires on 30% of all closings. That last claim does not survive the clean corpus and is withdrawn in Table `eftab:clean`.

Per class, precision ranges from 0.30 (`idle`) down to 0.07 (`wait`) and 0.09 (`errand`) – two classes that fire *below* the base rate and are therefore worse than not firing. But demoting classes barely moves the shipped number, from 0.152 to 0.159 under a greedy search over all nine, because the judge fires on almost everything the patterns do and more.

Nor does the judge’s confidence rescue it. Sliced into four bands, precision is 0.44 ($n = 9$), 0.16, 0.10 and 0.16 with increasing confidence: not monotonic, and flat everywhere the volume is.

We want to be careful about what this does and does not show. The label is one-sided: a developer often absorbs a premature stop and simply answers, which counts as NEW and caps measurable precision below its true value. So 0.15 is a floor rather than an estimate. What survives that caveat is the comparison of rates. The failure occurs on roughly one closing in

seven and the gate intervenes on three in four. Whatever the true precision is, the gate is firing several times more often than the thing it is looking for happens.

5.8 Tuning the judge against the label

With a label in hand the judge’s instruction becomes testable rather than a matter of taste. Its final line read *when unsure, answer STOP*, which is a plausible default for a gate whose false negatives cost a round trip. It is also, on this evidence, the reason the judge fired on two closings in three.

We ran three system prompts over the 747 clean closings, changing only the closing instruction and holding the model, the seed and the examples fixed.

Prompt	Final instruction	Fires	Prec.	95% CI	Rec.	F1
base	when unsure, answer STOP	469	0.117	[0.091, 0.150]	0.705	0.201
loose	when unsure, answer OK	412	0.124	[0.095, 0.159]	0.654	0.208
strict	a test, then OK when unsure	410	0.132	[0.102, 0.168]	0.692	0.221

Table 4: Judge prompt ablation over the 747 clean closings.

Flipping the default on its own moves the volume and not the quality. The judge fires 57 fewer times out of 747 and precision moves by 0.007, well inside the intervals. This is the third instruction in this work that a 9B model would not hold, after the background-work exception and the request for a confidence rating, and it is the cleanest demonstration: the sentence was changed to its opposite and the behaviour barely moved.

Replacing the default with a test the model can apply moves more. The *strict* prompt asks whether there is a concrete action the agent could take right now, in this repository, without information only the developer has; if what remains needs a preference or a priority, that is OK. It beats *loose* on every axis at once - fewer interventions, higher precision, higher recall - so between those two there is no operating point to choose.

Against *base* the comparison is a trade and we report it as one. Strict fires 59 fewer times and lifts precision from 0.117 to 0.132, and it pays one push for it: recall falls from 0.705 to 0.692. On the contaminated corpus this same comparison looked like a clean win on all three axes, which it is not. The intervals overlap throughout. What survives is the ordering, stable across both corpora: a criterion beats a disposition, and the gap between the two dispositions is noise.

5.9 Removing an exemption

The same label settled a design question the system had answered by intuition. Turns waiting on work the agent had launched were exempt: work in flight, no pattern raised, turn released without consulting the judge at all. The reasoning was that the harness wakes the session when the work finishes, so waiting is the mechanism rather than a stall.

Over the transcripts, 56 turns took that exemption and 12 of them ended with the developer pushing: 21% against a base rate of 13.9%. The exemption was releasing turns *worse* than average, and it was the single most confidently held assumption in the design.

It is gone. Waiting no longer excuses a turn; it selects a different request, which points out that the launched work reports back on its own and asks for the next piece in the meantime.

Removing it from the text was not enough. The waiting request lived inside the branch the judge reached only when it answered STOP, so a report written while a launched task ran was still released on the OK path. The system’s own log caught this: a summary closing went out at judge confidence 0.976 with work in flight, and the developer had to push.

The label sized the leak. Of 119 closings with work in flight, 22 ended in a push (18.5%) against 14.8% for closings with nothing running. Of the 37 that the judge released with OK, 6 still needed a push: 16.2%, above the base rate for the population. The OK path on a waiting turn was worse than average, the same finding as the original exemption one level down.

Waiting now decides the block on its own, whatever the judge answers. On the labelled corpus this raises recall from 0.700 to 0.764 at unchanged precision 0.150: 44 more fires, 9 of them real. The cost is bounded by the request itself, which accepts “nothing else can move without this result” as an answer and spends one turn collecting it.

5.10 A false positive the gate found on itself

The gate blocked the turn that reported the fix above. The matched phrase was `is (running|reviewing|writing) now`, and the sentence it matched was “Nothing is running now.” The pattern had fired on a clause asserting the opposite of what it looks for. This is the failure mode behind the `wait` class measuring 0.07 precision, visible as a single instance because the system logs what it matched in the agent’s own words rather than the regex.

Patterns now skip a match governed by a negation within four words, stopping at the nearest clause break, so a distant negation in the same message still leaves the phrase live. The distinction matters: “Nothing is running now” must not fire, while “I did not finish the audit; I will report back when it lands” must.

Over the 854 closings, 37 change labels and 3 of those were real pushes; 19 lose every firm label and fall through to the judge, which still sees them. The guard removes matches that were wrong by construction and costs almost no recall, because a negated stall phrase is rarely a stall.

5.11 Separating two questions the judge had been answering as one

Every result above treats the judge’s OK as a release. That conflates two questions. The judge is asked whether the agent stopped early, and a closing with nothing wrong in it passes that test while still leaving work available. The 12 pushes the shipped gate misses on the clean corpus are almost all of this shape: correct reports of finished work that a next action would have followed.

An OK therefore no longer closes the turn on its own. It asks once per chain whether a next action exists, and the release comes on the following pass: the turn ends when the agent looked and could not name one. The question is put only when the turn’s own tool calls name a candidate, which is the shape the block grading later forced and is measured below. The design point is that the second question is put to the worker, which has the repository in front of it, rather than to a 9B classifier reading a paragraph.

Of the 747 clean closings, 191 are released today and so become candidates for the question; 12 of those were real pushes. Because a closing now takes either a block, the question, or a release earned by having changed nothing, the recall ceiling rises with the coverage of the candidate reader and the binding constraint moves from detection to the worker’s own answer. The cost is one turn per clean closing, which the request bounds by accepting “I checked these and nothing remains” as a closing answer.

5.12 Before and after the gate

The corpus splits at 2026-08-28, when the gate was installed. Under a strict regular expression for an explicit correction, pushes fall from 10 of 664 ungated closings (1.5%) to 1 of 190 gated

ones (0.5%): the direction the gate is for, and not significant, with a two-sided Fisher exact test at $p = 0.47$.

Under the model-judged label this paper uses everywhere else, the effect disappears. 611 ungated closings carry 64 pushes (10.5%) and 136 gated ones carry 14 (10.3%), for $p = 1.0$. The two labels are counting different things - one an explicit rebuke, the other any reply that asks for work the agent could have done - and they disagree about whether anything changed.

We report the second as the answer, because it is the label the rest of the paper is scored against and because it is the larger sample. It says this study does not detect an effect of the gate on how often the developer has to ask. The comparison is observational either way: the eras differ in what was being worked on, and the gated era is the one where the gate was being built, which is not a neutral workload. What the exercise establishes is the size a future study would have to be - with a base rate near ten percent and a halving to detect, several hundred closings per arm rather than the 136 we have.

5.13 The judge's confidence

These counts are from the first pass. The rebuilt corpus stored verdicts without their token probabilities, so the distribution was not recomputed; the builder now keeps the probability with the verdict, so the next rebuild answers this without a second pass over the model. Taking the probability the judge assigns to its own verdict token, the distributions overlap heavily: median 0.95 for STOP ($n = 569$) and 0.87 for OK ($n = 272$). 115 STOP verdicts and 92 OK verdicts fall below 0.75.

At the extremes the signal is real. On hand-checked cases it reads 0.26 to 0.53 on messages that are finished reports and 0.97 to 1.00 on genuine premature stops. But on the 30 closings that reach the judge carrying doubts alone, it stops 24, and among the 6 stopped below 0.75 exactly three are finished reports and three are real cuts. A threshold there buys one correct release and pays for it with one lost catch.

That analysis assumed the only alternative to blocking was releasing, and under that assumption the conclusion held: the confidence was logged and used by nobody. A signal can be genuine, reproducible, and still not be a decision rule when both outcomes it chooses between are expensive.

The proactive question changed the arithmetic by adding a third outcome that is cheap when wrong. A verdict the judge cannot support no longer has to choose between accusing the agent and waving it through; it can ask. With that outcome available the threshold moves down to where it costs nothing: every block in the production log scores above 0.5, and over the 17 held-out messages the judge labels correctly its lowest score is 0.846. The single verdict below 0.5 was a false accusation, scoring 0.228 on a message that closed cleanly and had committed its work.

We report this as a caution about our own earlier reading. The threshold was not unusable; the two outcomes we had to spend it on were. Widening the action space, rather than sharpening the signal, is what made the signal actionable.

5.14 What the gate costs the machine it runs on

A gate that runs on every turn is a resident cost, and we did not measure it until it stopped the machine. Each stop started nine Python interpreters, one per checker, and the dead-code checker asked git once per symbol the turn had added, up to 120 processes on a single closing. On Windows a console program started from a console-less parent gets a console of its own, so the cost was visible: consoles opening and closing over the editor while the developer worked.

Three changes removed it. The checkers share one interpreter, keeping the contract that a checker is a file dropped in a directory. The git questions are asked once each and batched,

sixty symbols to a call, and only when the closing claims delivery: a turn that reads code and a turn that is still iterating ask git nothing at all. The measured cost of a stop is now 0 processes while iterating and 3 when the agent says the work is done, against 9 to 129 before. The suite fell from 133 processes to 42, and the remaining 42 are the checkers under test doing the thing they are tested for. That count is now defended rather than remembered: the runner keeps a ledger of every child any suite starts, and the last test in the suite reads the total and fails past a budget. A number that broke the machine once is a number a test should own.

The judge is the other half. An earlier version started the Ollama daemon when it found none, which loads a 5.5GB model server on a machine that never asked for one. On 2026-08-29 it did that on a busy workstation during a test run that had outlived its own cancellation; the machine ran out of memory and had to be restarted. The gate now starts nothing, and the tests that need the model are opt-in, like the git ones. This is not a footnote about our hardware. A guard that degrades the machine it guards will be switched off, and a gate that is switched off has whatever accuracy you like.

5.15 The corpus rebuilt on clean replies

Every number above this point was computed on a corpus that included the system’s own output. Three contaminations were found and are described in Section 5.5: the gate’s wake read as a developer reply, bare image placeholders, and harness notices. The corpus was rebuilt from the same 66 transcripts with those readers corrected. The lane and class ablations are re-run on it below; the headline figures move as follows.

	contaminated	clean
labelled closings	841	747
pushes	117	78
base rate	13.9%	10.4%
gate fires	624	556
precision	0.15	0.119
recall	0.81	0.846

The direction is the one contamination predicts. The rows removed were mostly positives the system had manufactured for itself, so the base rate falls by a quarter and precision falls with it, while recall rises slightly. The qualitative finding is unchanged and sharper: the gate fires on 74% of all closings to catch 85% of the 10% that matter.

Every configuration was then re-run on the clean rows. Nothing in this table needs the model: the corpus stores each lane’s opinion per row, so the whole ablation is arithmetic over stored labels.

The intervals are the honest reading. Every lane configuration covers the base rate of 0.104, so on this corpus no configuration is distinguishable from firing at random. The clean numbers therefore withdraw a claim made above. On the contaminated corpus, requiring both lanes to agree reached 0.21 against a base rate of 0.139 and was described as the one configuration that clearly beats chance. On clean rows the same configuration reaches 0.130 against 0.104. The gap is 26 closings out of 747 - it survives as a direction and not as a result, and no configuration we measured separates itself from the base rate. The ordering holds: agreement between lanes is the most precise setting and the shipped either-lane rule the least, trading precision for the recall it was tuned for.

The per-class picture holds too, with sharper extremes.

Lane configuration	Fires on	Prec.	95% CI	Recall	F1
Base rate (the failure itself)	78	0.104	[0.084, 0.128]	1.000	—
Firm patterns only	389	0.123	[0.094, 0.160]	0.615	0.206
Any pattern	515	0.117	[0.092, 0.147]	0.769	0.202
Judge only	460	0.122	[0.095, 0.155]	0.718	0.208
Firm or judge (shipped)	556	0.119	[0.094, 0.148]	0.846	0.208
Firm and judge	293	0.130	[0.096, 0.173]	0.487	0.205

Table 5: The same ablation on the clean corpus of 747 closings, with Wilson intervals on precision. Every interval covers the base rate.

Class	Fires on	Precision	95% CI
ask	177	0.113	[0.074, 0.168]
shape	137	0.197	[0.139, 0.272]
announce	128	0.109	[0.066, 0.175]
wait	88	0.034	[0.012, 0.095]
handoff	43	0.116	[0.051, 0.245]
errand	40	0.050	[0.014, 0.165]
idle	35	0.257	[0.142, 0.421]
defer	35	0.086	[0.030, 0.224]
read	15	0.267	[0.109, 0.520]

Two classes separate from the base rate upward: **shape** at 0.197 on 137 closings and **idle** at 0.257 on 35, both with intervals clear of 0.128. **shape** is the one class written without a lexicon - it fires on a closing that ends in a question mark or on a short turn that ran nothing - and it is the most precise rule in the system on the volume that matters.

One class separates downward. **wait** fires 88 times at 0.034, a third of the base rate, and its interval reaches 0.095. **errand** sits with it at 0.050 on 40 closings on a wider interval. A rule that fires at a third of the rate of the thing it looks for is worse than not firing, and **wait** was the fourth most common intervention the gate made.

The same corpus names what the gate misses. Twelve pushes get past both lanes, and reading them takes minutes: most are a turn that ends by naming the very next move in the present tense, in either language, and one ends on a colon with the thing it promised never written. Three rules were added from that reading - six Spanish verbs onto an existing line, a postponed recheck, and a shape rule for a closing whose last line ends in a colon - and measured on the same rows. The gate goes to 536 fires, precision 0.125, recall 0.859: fewer interventions, higher precision and higher recall than the shipped configuration at once. Both closings the new rules add are pushes.

We are careful about what that shows. Rules read off a corpus and scored on the same corpus are fitted to it, and two closings is not evidence. What the loop demonstrates is the cost: an afternoon, no annotation, and a list of twelve messages the developer had already labelled by answering them.

5.16 Does the tuning transfer?

Every choice above was searched on the rows it is reported on, which is the objection a reader should raise. The corpus answers it: closings carry the session they came from, so it splits by session rather than by row. We run the greedy class demotion on half the sessions and score the classes it chose on the other half, twenty times.

The search transfers in direction and barely in size. Held-out precision rises in 16 of 20 splits, and the mean gain is $+0.003$ - about two closings. The demoted set is not stable either: the search removes between four and seven classes depending on the half it sees. Three classes are removed in all twenty splits (`ask`, `idle`, `errand`), `wait` in 17 and `shape` in 11 - and `shape` and `idle` are the two classes whose precision is significantly above the base rate. A search that maximises precision over the union with the judge will happily remove a good pattern whose rows the judge already covers.

That is the honest ceiling of pattern tuning on a corpus this size. The classes the search agrees on are the ones the judge covers, so removing them costs nothing and buys almost nothing, and everything past that is fitting. It also says what a larger corpus would be for: not a better score, which we would not trust, but a demotion set that stops changing when you change the half.

Demoting classes still barely moves the shipped number. A greedy search removes six of the nine and lifts precision from 0.119 to 0.127, because the judge fires on nearly everything the patterns do. The pattern list cannot be tuned out of this: the two lanes are not independent, and removing one lane's worst rules leaves the other lane firing on the same rows.

The two classes below the base rate were demoted anyway, from firm to doubt: they no longer block on their own and now only raise a question the judge can settle. The effect is measured and modest. The pattern lane drops from 389 fires to 326 and its precision rises from 0.123 to 0.132, at the cost of one push out of 78. The whole gate moves from 556 fires to 534 with recall from 0.846 to 0.833. We report the change because a rule that fires 128 times at a third of the base rate is worse than no rule, and because the size of the gain is the point: 63 fewer interventions and a precision still inside the noise of a corpus this small.

5.17 Two changes that did not earn their place

Both were written, wired behind a switch, measured on 150 clean rows, and left switched off.

The judge reads the closing and the single developer message it answers. A reply that continues something agreed two turns earlier is hard to read that way, so we gave the judge the last three and four exchanges, each a developer request with the answer the agent gave it, excluding the working in between. Precision moved from 0.245 to 0.248 at three turns and to 0.243 at four; recall held at three and fell at four. The window buys nothing.

The judge also cannot see what the turn did: a message claiming a green suite is the same string whether the agent ran it or imagined it. We passed the count of tool calls the turn made as a fact rather than asking the model to infer one. Catches were identical and false positives rose from 70 to 74. The count alone is not the fact that matters; which tools ran probably is, and that is a larger change than this experiment tested.

We report both because the alternative is a paper in which every idea worked.

5.18 What a block buys

Precision against the push label asks whether the gate was right. It does not ask whether the block helped, and those are different questions: a block that fires on a closing the developer would have accepted still buys work if the agent goes and does something. The system labels this for itself. A block wakes the agent, and the tool calls it makes before the developer next speaks are the answer.

Over 41 graded blocks, 35 bought work and 6 bought none: an 85% conversion, with a 95% interval of $[0.716, 0.931]$. The interval is wide and clear of a half: the interventions are mostly bought, whatever the exact share. The rate is not uniform across lanes.

Lane	Blocks	Bought nothing
judge	11	9%
waiting	10	0%
announce	7	14%
announce?	7	14%
proactive	6	50%
ask	4	0%
wait	3	33%

The lanes are not equal. The judge wastes one block in eleven, the pattern lanes about one in seven, and the release for work in flight - which tells the agent to keep going while a background job runs - wastes none of its ten. The proactive question, the newest lane and the one that fires on closings nothing is wrong with, wastes one block in two - by a distance the worst in the system, and the cost of putting a question to every clean closing.

So the question was made specific. It now names what the turn’s own tool calls show it left behind: source files written with no test written beside them, code changed with nothing in the documentation moving with it. The items are read off the record rather than inferred, which makes a denial answerable with evidence instead of a shrug. Whether that lifts the conversion is the measurement to make once enough blocks have accumulated; we report the mechanism and the baseline it has to beat.

The count then decided the lane’s shape. A candidate exists on 30% of 1291 real turns, so a question that always fires is a question with nothing to point at two times in three. The gate now releases those turns instead. The proactive lane asks when the transcript names something and stays quiet otherwise, which removes about seven interventions in ten from the lane with the worst conversion in the system and leaves every remaining one specific.

5.19 Verifying claims instead of reading intent

Every measurement so far scores the gate against one weak label: whether the developer pushed back. That label answers whether a human had to ask, and a false claim only enters it when the human noticed. A claim can be checked without a label at all. The agent wrote that it committed; the turn either ran a commit or it did not, and the transcript holds both halves.

We extracted first-person delivery claims from every closing and matched them against the commands the session ran. Of 43 closings claiming a commit, 1 has no commit anywhere in the session. Of 83 claiming a green suite, 5 have no test run. Scoped to the turn that made the claim rather than the session, the numbers are 6 and 29, which measures reporting on evidence gathered earlier rather than a claim that is untrue.

A first version of the extractor counted the word commit anywhere and read 41% of claims as contradicted. That number was a bad regular expression, not a finding: an agent naming somebody else’s commit, or offering to make one, was scored as claiming it had. We report it because the correction is the whole lesson - a verification lane is only as good as the claim extractor, and a loose extractor manufactures the finding it is looking for.

The conclusion is negative and useful. Contradicted claims occur on 0.5% of closings, so this lane cannot lift precision: there is almost nothing there to catch. The agent in these transcripts overstates what it did far less often than it stops with work available, and those are separate failures needing separate mechanisms.

5.20 What each lane contributes that the other does not

The lanes overlap heavily, so their separate precisions say little about whether both are worth running. The useful cut is the rows where one lane fires alone.

Firm patterns fire without the judge on 76 closings and 11 of those are pushes, a precision of 0.145 against a base rate of 0.104. The judge fires without any firm pattern on 207 closings and 22 are pushes: 0.106, which is the base rate to three decimals. The expensive lane’s unique contribution is volume - it triples the fires nobody else catches - at exactly chance precision, and it is where the recall comes from.

The doubt patterns cost less than they appear to. On 65 closings a doubt fires and the judge then passes, so nothing blocks; 4 of those are pushes, a rate of 0.062. A doubt that only raises the message to the judge is cheap in interventions and cheap in mistakes, which is the argument for demoting a class instead of deleting it.

5.21 The configuration this work leaves behind

The changes above are small and they are in the same direction, so it is worth stating what the gate is when the paper ends.

Configuration	Fires on	Prec.	95% CI	Recall
At corpus build	556	0.119	[0.094, 0.148]	0.846
Two classes demoted	534	0.122	[0.097, 0.153]	0.833
Three rules added	536	0.125	[0.100, 0.156]	0.859

The gate fires 20 fewer times out of 747, catches one more push, and its precision interval still covers the base rate. The pattern lane on its own moves further than the gate does, from 0.123 to 0.137 [0.104, 0.178] on 329 fires, which is the most precise configuration in this paper and still not separated from a base rate of 0.104. That is the whole shape of the result in one line. Nobody should read this as the gate having been fixed. What it is is a loop that ran: a label nobody wrote, a measurement, two changes the measurement asked for, and the same measurement repeated. The interesting figure is not 0.125. It is that both changes were decided in an afternoon, by a person reading twelve messages and a table, on data that was already on the disk.

The proactive question is outside this table because the push label cannot score it: it fires on closings the developer accepted, which are negatives by construction. It is scored by the second label instead, and it is the lane with the most room left.

6 The self-labeling loop

Every decision the gate makes is appended to a private log: the lane that decided, the patterns that fired, the judge’s verdict and confidence, the latency, and the first line of the message. The log records what was decided. It does not record whether the decision was right, and for the first weeks tuning meant reading transcripts by hand.

The missing label was already in the transcript, a few lines below the block. A block wakes the agent, and from there exactly one of two things happens: the agent goes and does work, or it writes another paragraph and stops again.

Grading rule. The number of tool calls between a block and the next time the user speaks is the label for that block. Above a threshold, the block bought work. At or below it, the block bought a paragraph.

Nobody supplies this. It is read from the same transcripts the gate already opens. The report joins each logged decision back to its transcript, counts the intervening tool calls, and aggregates by pattern class and by judge confidence. When a class accumulates enough blocks and most of them bought nothing, the report names that class for demotion from firm to doubt.

The report runs at session start and stays silent unless a class is ripe, speaking at most once per day. A report that speaks on every session is a report that gets skipped, and the loop’s output is consumed by an agent with the same attention economics as its user.

The first two blocks the loop graded had bought zero tool calls each, on the same pattern class that had been demoted by hand that morning after measuring 888 closings. The loop reproduced a hand measurement in one second.

7 Negative results

These cost more than the positive ones and generalize better.

The judge cannot report its own confidence. Asked to append a confidence rating to its verdict, the 9B model answered with the maximum every time, across every message we tried. The number was constant and therefore carried nothing.

Self-consistency did not separate either. Re-sampling the same message five times at temperature 0.8 produced five identical verdicts, including on messages the verdict got wrong. At this scale the model is confidently wrong in a stable way, so disagreement-based confidence [7] has nothing to measure.

Token log-probabilities did move. Taking the probability the model assigned to the single verdict token it emitted produced a number that varies with the message, and that separates at the extremes. It is the only one of the three that worked.

A 9B judge cannot hold an exception. Nineteen percent of the judge’s blocks were turns in which the agent had correctly launched background work and was waiting on it. We tried to teach this in the prompt, first as prose and then as few-shot examples. Both failed: the model could not hold the exception against its own standing instruction to answer STOP when unsure, and the examples degraded unrelated verdicts. The fix was to remove the question. Waiting is now established deterministically from the transcript, before the judge is called at all. *An instruction a small model cannot hold is a decision that belongs outside the model.*

Nor a disposition. The judge’s instruction ended with *when unsure, answer STOP*. Replacing it with its exact opposite, *when unsure, answer OK*, changed 31 verdicts out of 841 and moved precision by 0.001. Replacing it instead with a test the model can apply – is there an action available right now that needs nothing only the developer knows – moved precision, recall and intervention count together. Three instructions in this work went unheld by the same model, and the pattern across them is consistent: it follows a criterion it can evaluate against the text in front of it, and ignores a disposition about how to feel when unsure.

A shared accelerator breaks reproducibility. Two test suites run concurrently produced different verdicts on byte-identical input. The cause is request batching in the serving daemon. We first misdiagnosed this as model flakiness and then as a prompt defect, and “fixed” it

by adding an example, which made it worse. Determinism required a fixed seed and a machine-wide lock.

An A/B harness must pin everything the code reads. Our first before/after comparison reported three regressions. Both arms had been given different working directories, so one checker saw files on one side only. The regressions vanished once the directory was pinned.

8 Threats to validity

One user, one idiom. The corpus comes from a single developer. The patterns carry that developer’s phrasing, and the judge required an explicit reading list for River Plate Spanish because *quedó cableado* (“it is wired end to end”) was being read as work in progress. Nothing here has been tested on a second user’s transcripts.

The gate contaminates its own corpus. Gated transcripts were produced with the gate running, so their closings are partly the gate’s own output distribution. This is why all label-based evaluation is restricted to ungated transcripts, and why the era comparison in Section 5 is observational rather than controlled.

The push label is one-sided. Absence of a push is not evidence the turn was complete: a developer often absorbs a premature stop and simply answers. Every precision in this paper is therefore a floor on the true value and not an estimate of it. What the label supports is the comparison of rates, and the comparison holds under any one-sided correction: the gate fires several times more often than a push occurs.

The grading rule rewards motion. Counting tool calls after a block cannot see whether the work was any good. An agent that learned to emit cheap tool calls after every block would defeat it. We have not observed this, and it is the most obvious way the loop could be gamed.

Rules are searched on the rows they are scored on. Every class demotion, and the three rules added from the missed pushes, were chosen by reading this corpus and reported on it. Section efsec:results splits the corpus by session and scores the search on held-out sessions to bound the damage: the direction transfers, the size is about two closings, and the set of classes chosen is not stable across halves.

The corpus is small where it counts. 747 closings carry 78 pushes, so every precision here has a Wilson interval two to four points wide and every per-class number rests on tens of rows. No configuration we measured separates from the base rate at 95%. Conclusions in this paper are stated at the level the intervals support, which is rates and directions rather than values.

Thresholds are tuned, not derived. The demotion threshold and the work threshold were chosen for the volume this log receives.

9 Discussion

Three things generalize beyond this system.

Put the decision where the capability is. The clearest single result is the background-work case: an exception that a 9B model could not hold in its prompt became trivial once it was decided by ten lines of transcript inspection. The temptation with a model in the loop is to route every judgement through it. The cheaper lanes should take everything they can prove, and the marginal cut says how much that is worth here: on the closings only the patterns catch, precision is 0.145; on the closings only the judge catches, it is 0.106 on a base rate of 0.104. The model earns its place by covering three times the volume, and it earns none of it by being more discriminating than a regular expression.

Confidence is a router before it is a decision. Per-pattern confidence is what makes the local model affordable, because it decides which messages are worth a model at all. The judge’s own confidence was informative and unsafe to threshold, because dropping a low-confidence STOP loses real pushes. It became usable when the action space grew a third option: a STOP under the floor is put as the question instead of as an accusation, and the turn is neither blocked nor released. A weak signal that cannot pick between two actions can still pick a third.

Interventions leave labels. Any system that interrupts an agent gets a natural experiment for free: the agent’s behaviour after the interruption. We suspect this generalizes to most guardrails. The evaluation problem for interventions is often not a missing label but an unread one.

10 Limitations and future work

The loop corrects unevenly. It finds patterns that block too much cheaply, because every block leaves a graded trace and the report reads itself. The other side needs the push label: a turn the gate allowed and the developer then pushed back on is a miss, and reading the twelve of them took an afternoon and produced three rules. That path is manual, and it is only trustworthy on ungated transcripts, where the label is not partly the gate’s own output. On this corpus the label does not thin out under the gate - 10.3% of gated closings carry a push against 10.5% of ungated ones - so the constraint is contamination rather than sparsity, and it is the same constraint either way: the misses can only be read where the gate was not running.

Three extensions follow directly. Demotion is currently proposed and applied by an agent reading the report; promotion in the other direction is not implemented. The judge is a general instruction-tuned model, and the graded log is exactly the dataset needed to fine-tune a specialist. And no part of this has been run against a second user, which is the first thing that should happen.

11 Conclusion

A gate that stops an agent from stopping early is easy to build and hard to evaluate, because evaluation needs labels and labels need a reader. Both labels this system needs were already lying in the transcript: the developer’s own reply before the gate exists, and the agent’s own behaviour after the gate intervenes. Neither costs an annotation.

Reading them was uncomfortable and useful in the same measure. The gate fires on three closings in four and is right about one in eight, which is the rate at which the failure occurs at all: no configuration we measured separates from it at 95%. The first thing the labels overturned was the design’s most confident exemption, which turned out to be releasing

turns worse than average. The second was a prompt written as a disposition, replaced by one written as a test, which improved every number at once.

Neither correction came from a new idea. Both came from finally measuring something that had been sitting in the transcript the whole time.

A Pattern classes

The 299 patterns are grouped into nine classes by what the closing message is doing. A class name written with a trailing question mark marks a doubt, which defers to the judge instead of deciding.

Class	Patterns	Reads as	Example trigger
announce	70+15	a step named instead of taken	“next I will”, <i>paso a</i>
ask	56+1	a question it may answer itself	“do you want me to”, <i>querés que</i>
defer	32+2	work made conditional on being asked	“if you want, I can”
wait?	0+33	deferring to the user’s clock	“I’ll wait”, “let me know when”
handoff	24	giving the work back	“over to you”, “your call”
errand?	0+23	sending the user to run something	“run this and tell me”
read	21+2	asking what the repo already answers	“where is the spec”
idle	12+1	reporting that nothing happened	“still running”, “no changes”
ack	7	a bare acknowledgement	“done”, “ok”

Table 6: The nine pattern classes. The second column counts firm patterns plus doubts, where a doubt raises the message to the judge instead of deciding. 222 of the 299 are firm.

Two classes need an exception that is decided outside the patterns. **ask** patterns are suppressed on long messages, because a long report that ends in a courtesy question is a report, not a request for permission. **wait?** patterns are suppressed when the transcript shows the agent is waiting on work it launched itself.

wait? and **errand?** are written as doubts because they fire below the base rate of the failure they look for: 0.034 and 0.050 against 0.104 (Table 5). They raise the closing to the judge and decide nothing on their own.

B The judge’s instruction

The judge’s system prompt is a decision rule, not a persona. Its shape, in order: the agent has standing permission and never needs to ask; a definition of STOP listing the observable behaviours; a definition of OK; a paragraph on the most common hard case, in which the message reports real finished work *and* names something still open, which is STOP; a counterweight stating that a closed caveat is not a pending item, because an agent that learns to delete its warnings to get past the gate is worse than one that stops; a note that deferred work often dresses as a disclosure; a note that a conditional offer is a stop; a short reading list for River Plate Spanish; and a test in place of a default: is there a concrete action the

agent could take right now without information only the developer has, with OK when unsure. That last line was measured against its two alternatives in Table 4.

The counterweight paragraph was added after the gate began punishing useful warnings. It is the only part of the prompt that argues against the gate’s own purpose, and it is load-bearing.

C The grading rule

```
for each logged block:
    find the blocked message in its transcript
    count tool calls until the user speaks again
    (a tool result is not the user speaking)
    fewer than 3 tool calls  $\Rightarrow$  the block bought nothing

for each pattern class:
    if it has 8 or more graded blocks
    and 70% or more bought nothing:
        name it for demotion
```

The threshold of three tool calls exists because a woken agent commonly reads one or two files to re-establish context before deciding to stop again. Zero is too strict and would count that re-orientation as work.

D Reproducing the numbers

Every table in this paper is produced by a script in the hook’s own repository, run against the transcripts on the developer’s machine. The corpus builder is the only one that calls the model; the rest read its stored labels and are arithmetic, so they run in seconds and give the same answer every time.

Script	What it does	Feeds
<code>corpus.py</code>	builds the labelled corpus; <code>-patterns</code> re-scores the pattern lane alone and keeps the stored verdicts	Table 5 inputs
<code>bench-lanes.py</code>	lane and class ablation with Wilson intervals	Table 5
<code>bench-split.py</code>	the same search scored on held-out sessions	transfer result
<code>bench-prompt.py</code>	three closing instructions over the same rows	Table 4
<code>bench-sure.py</code>	the judge’s confidence against the label	confidence bands
<code>bench-contradiction.py</code>	delivery claims against the commands that ran	claim verification
<code>judge-report.py</code>	grades each block by the tool calls it bought	block conversion
<code>test_paper.py</code>	fails when a number here stops matching the repo	all of it

The corpus is versioned by keeping the file from each build, so a table can be recomputed against the state it was written from. The builder takes about 13 minutes over 764 closings

on a consumer GPU, one model call per closing at temperature 0 with a fixed seed. The prompt ablation is three passes of the same shape. Nothing else here needs a GPU.

References

- [1] Y. Bai et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*, 2022.
- [2] S. Kadavath et al. Language Models (Mostly) Know What They Know. *arXiv:2207.05221*, 2022.
- [3] H. Lightman et al. Let’s Verify Step by Step. *arXiv:2305.20050*, 2023.
- [4] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. *NeurIPS*, 2023.
- [5] A. Ratner et al. Snorkel: Rapid Training Data Creation with Weak Supervision. *VLDB*, 2017.
- [6] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS*, 2023.
- [7] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023.
- [8] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.